

모두를 위한 딥러닝 (입문편)

허 찬

강의 계획표 - 모두를 위한 딥러닝 (입문편)

학습주제(강좌명)	차시(모듈)	차시별 주제	학습내용
모두를 위한 딥러닝 (입문편)	1	Dive into deep learning	머신러닝, 딥러닝, 인공신경망의 개념 이해 딥러닝의 분류 및 응용 분야 이해 딥러닝 모델의 장/단점 이해
	2	딥러닝 코딩 환경 준비	머신러닝/딥러닝 워크플로우의 이해 파이썬 기반 머신러닝, 딥러닝 프레임워크의 이해 패키지 설치 및 기초 벡터 연산 설명
	3	딥러닝 모델 학습의 4steps	딥러닝 학습 과정의 단계별 이해
	4	데이터 시각화	데이터 시각화 Pandas, Matplotlib, Seaborn 패키지의 사용
	5	딥러닝을 이용한 분류 모델 실습	딥러닝 코드 실습을 통한 분류 문제 해결
	6	딥러닝을 이용한 회귀 모델 실습	딥러닝 코드 실습을 통한 회귀 문제 해결
	7	딥러닝 모델의 성능 향상 기법	딥러닝 기본 모델의 성능 향상 기법

1.3	딥러닝 모델 학습의 4steps	딥러닝 학습 과정의 단계별 이해
-----	-------------------	-------------------

Contents

- 딥러닝 모델 학습의 4steps
- 예제를 통한 4steps 구현 (파이썬)
- 예제를 통한 4steps 구현 (케라스)

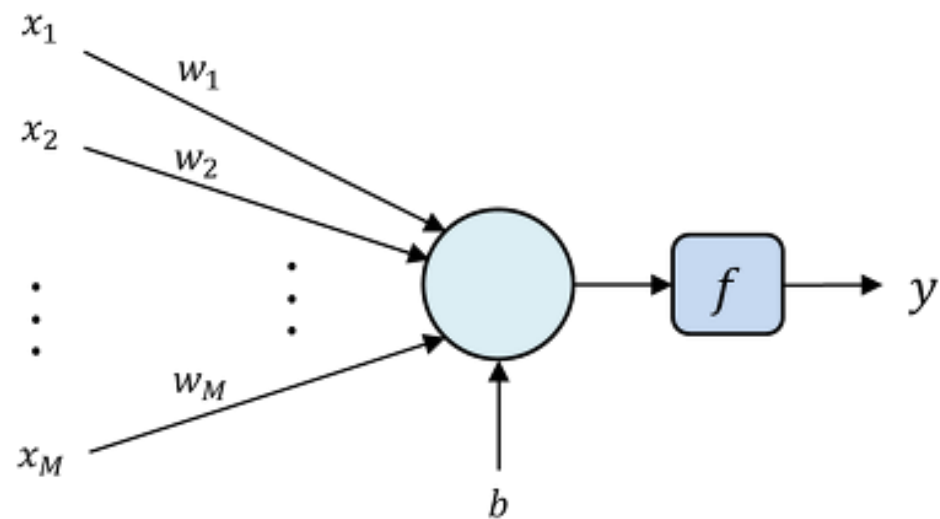
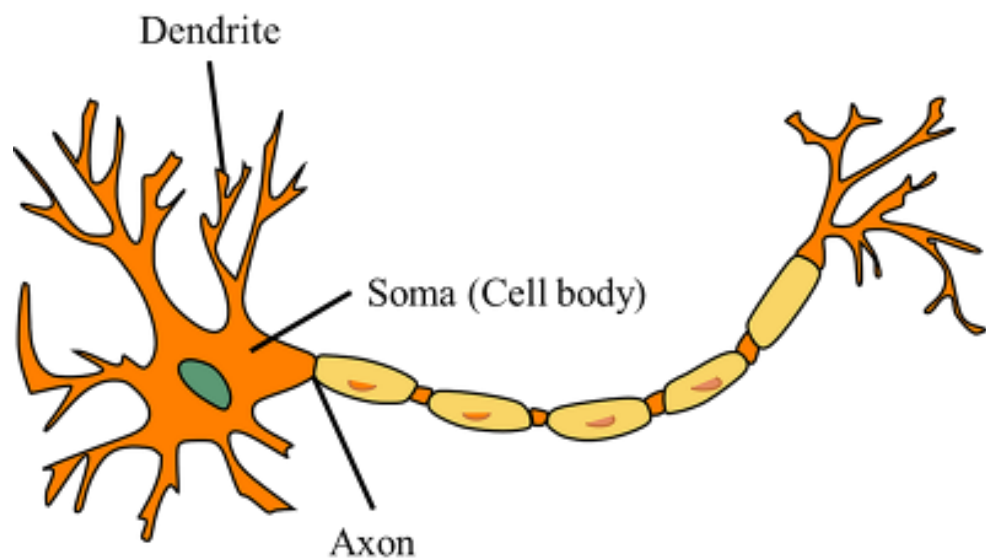
딥러닝 모델은 어떤 일을 하나?

딥러닝 모델은 어떤 일을 하나?

> > **행렬 연산을 통해 정답을 추론**

1.3 딥러닝 모델 학습의 4step

– A single neuron

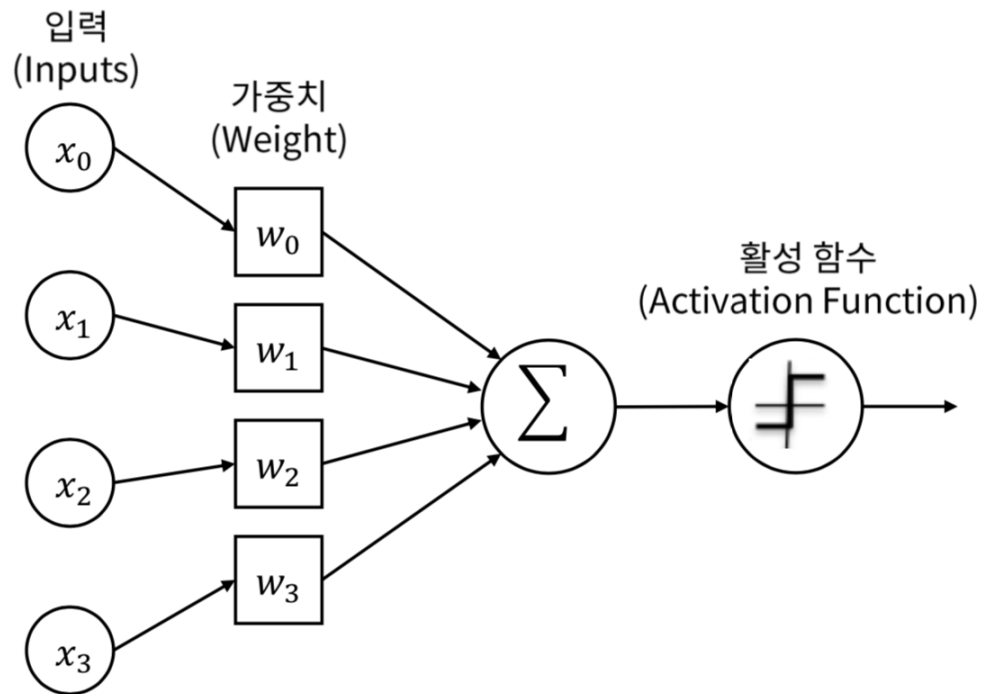
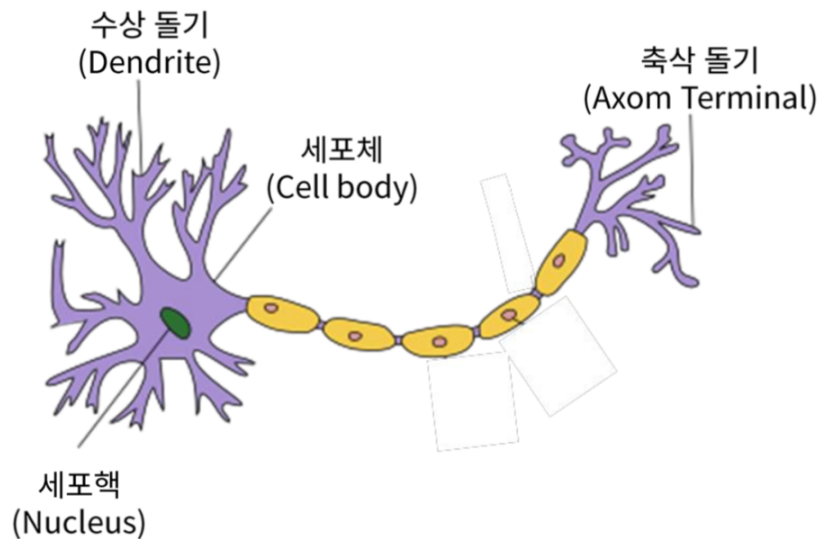


- 단일 신경 뉴런을 수학적으로 묘사한 구조

1.3 딥러닝 모델 학습의 4step

– A single neuron

신경 세포

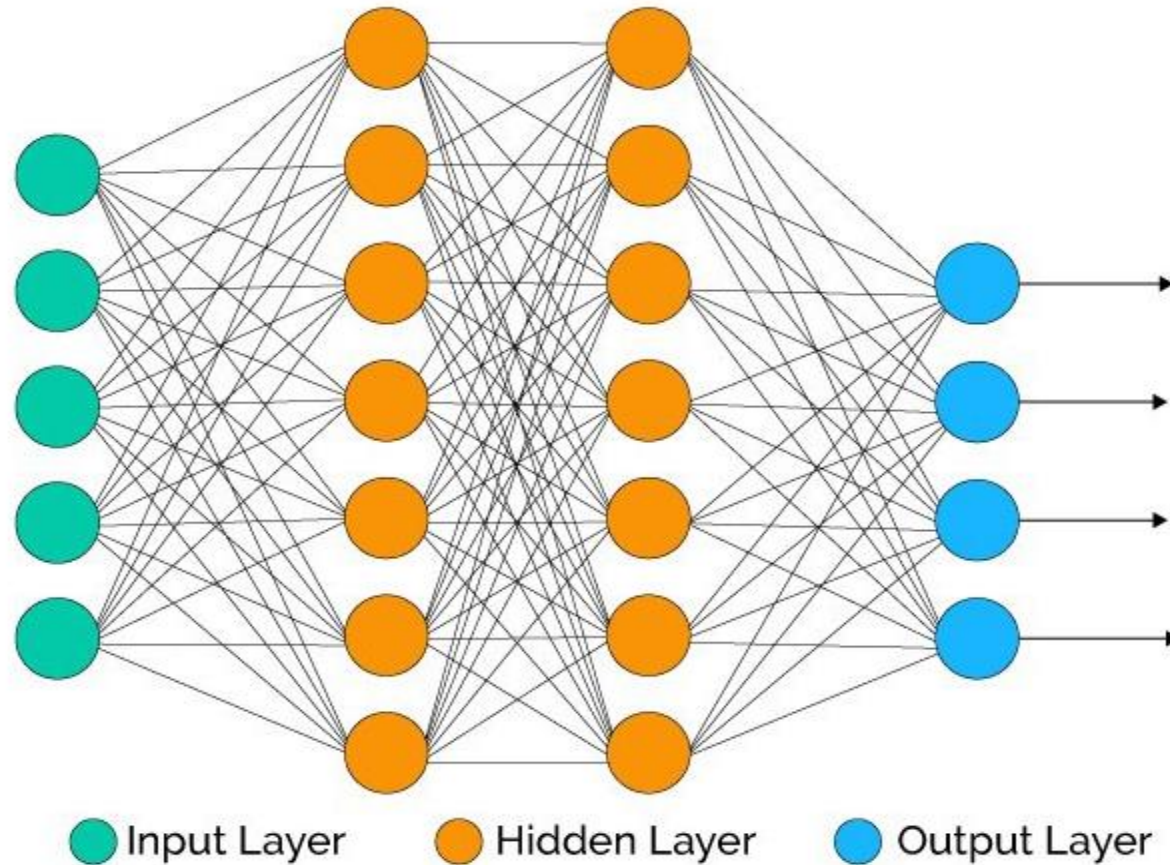


생물학적인 신경 세포를 단순화하여 모델링한 Neuron. 여러 신호를 받아, 하나의 신호를 만들어 전달하는 역할을 한다.

출력을 내기 전에 활성 함수(activation function)를 통해 비선형 특성을 가할 수 있다.

1.3 딥러닝 모델 학습의 4step

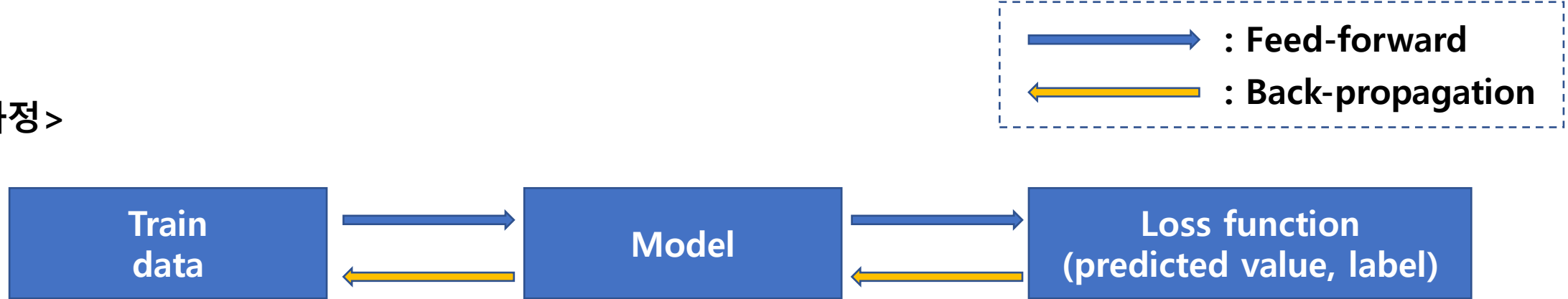
– Single neuron x N -> Deep Neural Network



• 다중 신경망 구조

1.3 딥러닝 모델 학습의 4step

< 학습과정 >



< 추론과정 >



1.3 Training 4 steps in deep learning

Training 4-Steps

1. Feed-forward
2. Loss calcuation
3. Backpropagation
4. Gradient update

1.3 Training 4 steps in deep learning

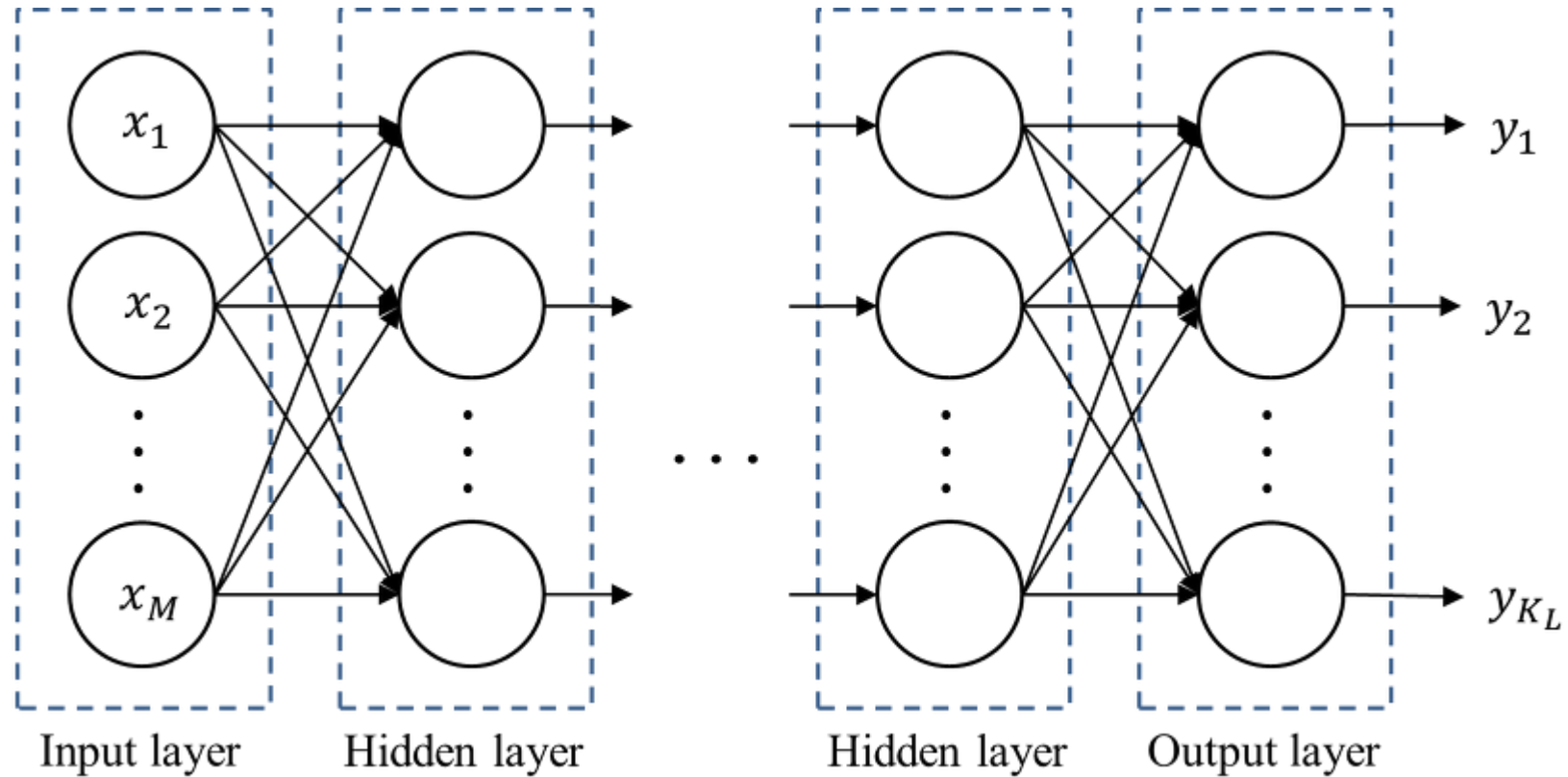
학습 – (1) Feed-forward

1. Feed-forward process:

- 모델이 데이터를 보고 의사결정을 내리는 과정.
- 입력 레이어에 데이터가 들어가면 연결된 중간 레이어(은닉 레이어)로 결과가 전달.
- 레이어 사이사이에 비선형 함수를 통하여 모델이 복잡한 패턴을 학습하도록 만듦.
- 최종 출력 레이어를 통해 신경망이 의사 결정을 내림.
- 입력>은닉>출력 레이어 순.

1.3 Training 4 steps in deep learning

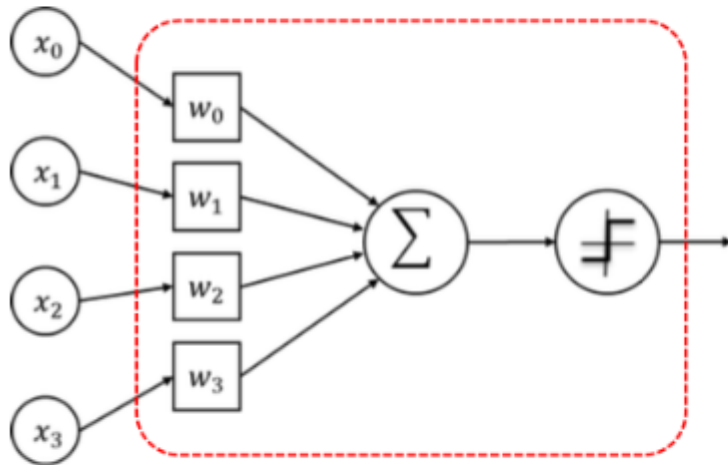
학습 - (1) Feed-forward



1.3 Training 4 steps in deep learning

학습 - (1) Feed-forward

뉴런의 수학적 표현



$$\sum_{i=0}^{N-1} w_i x_i = \mathbf{w} \cdot \mathbf{x} = \mathbf{w}^T \mathbf{x}$$

$$y = a \left(\underbrace{\sum_{i=0}^{N-1} w_i x_i}_{\text{두 벡터의 내적}} + b \right) \quad \text{편향(Bias)}$$

활성함수

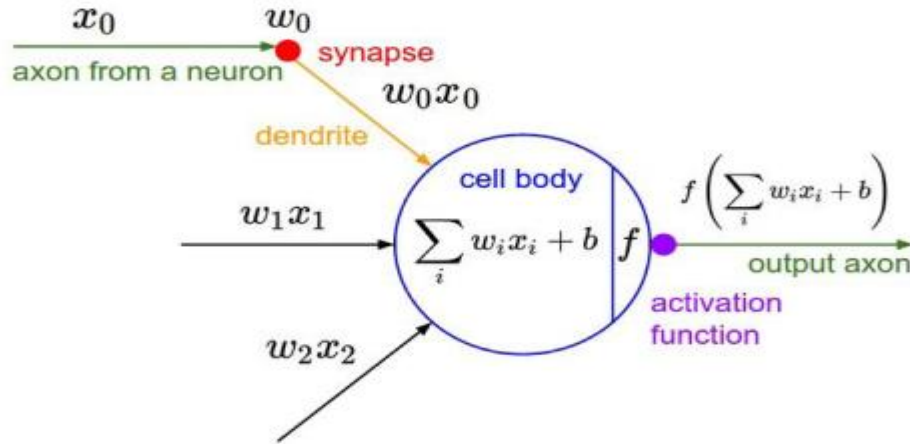
$$y = a(\mathbf{w}^T \mathbf{x} + b)$$

뉴런은 수학적으로 **두 벡터의 내적**으로 쉽게 표현할 수 있다.

1.3 Training 4 steps in deep learning

학습 - (1) Feed-forward (Activation function)

Activation Functions

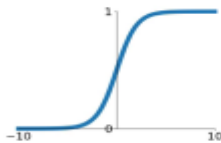


- 뉴런은 어느 정도 이상의 값을 신호로 통과
- 선형 레이어를 여러 개 쌓아도 하나의 선형 모델
- 뉴런의 출력 이전에 비선형성을 부여하기 위해 activation function을 통과

Activation Functions

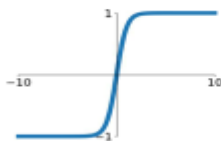
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



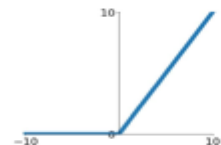
tanh

$$\tanh(x)$$



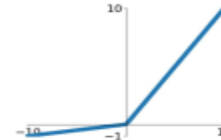
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

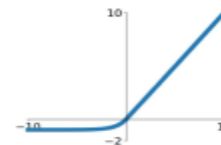


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



1.3 Training 4 steps in deep learning

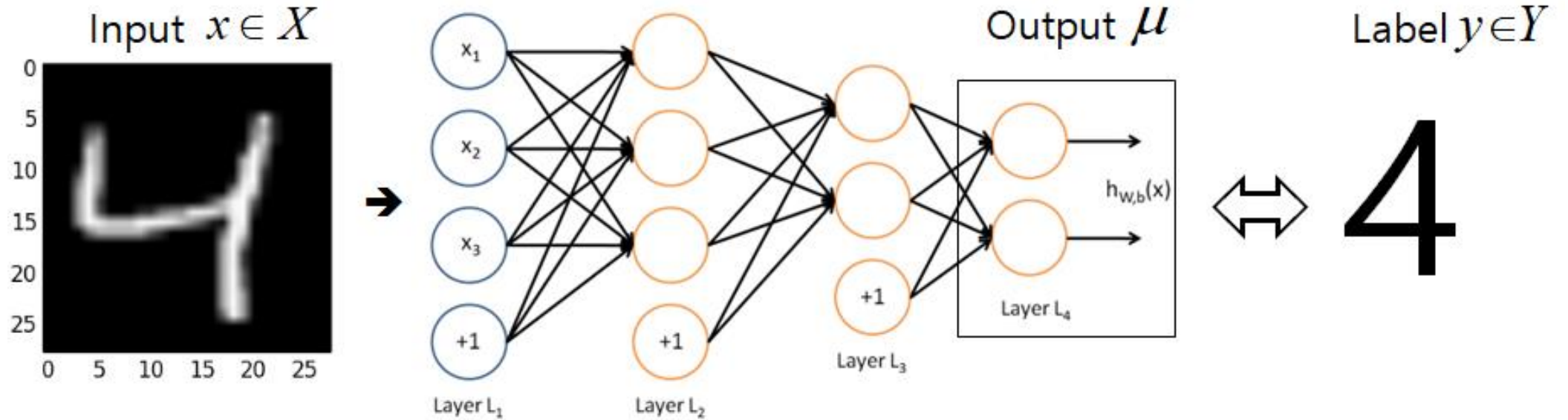
학습 – (2) Loss calculation

2. Loss calculation process:

- 출력 레이어에서 나온 결과를 미리 설정한 정답과 비교.
- 모델의 추론과 정답을 비교하는 과정
- 다양한 목적에 따라 변형 가능

1.3 Training 4 steps in deep learning

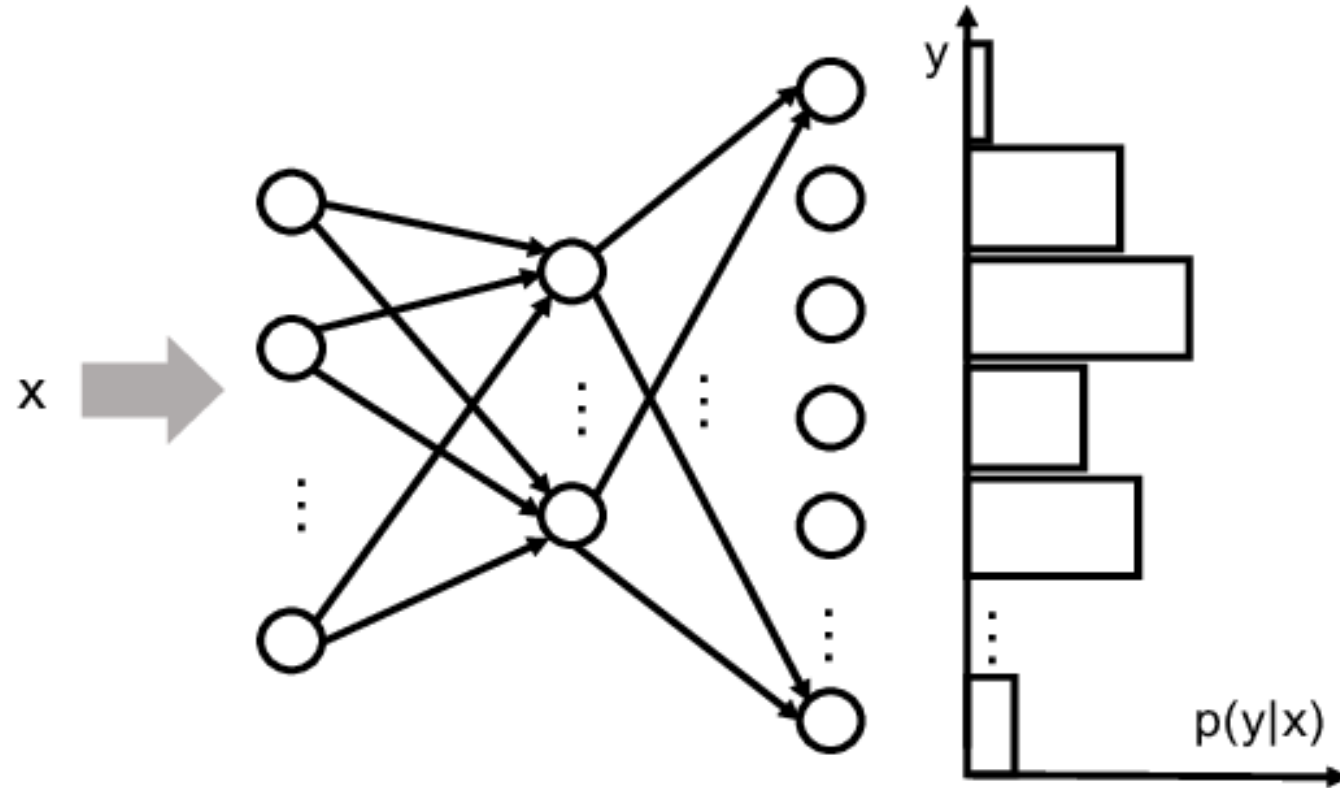
학습 - (2) Loss calculation



- 분류 문제의 경우

1.3 Training 4 steps in deep learning

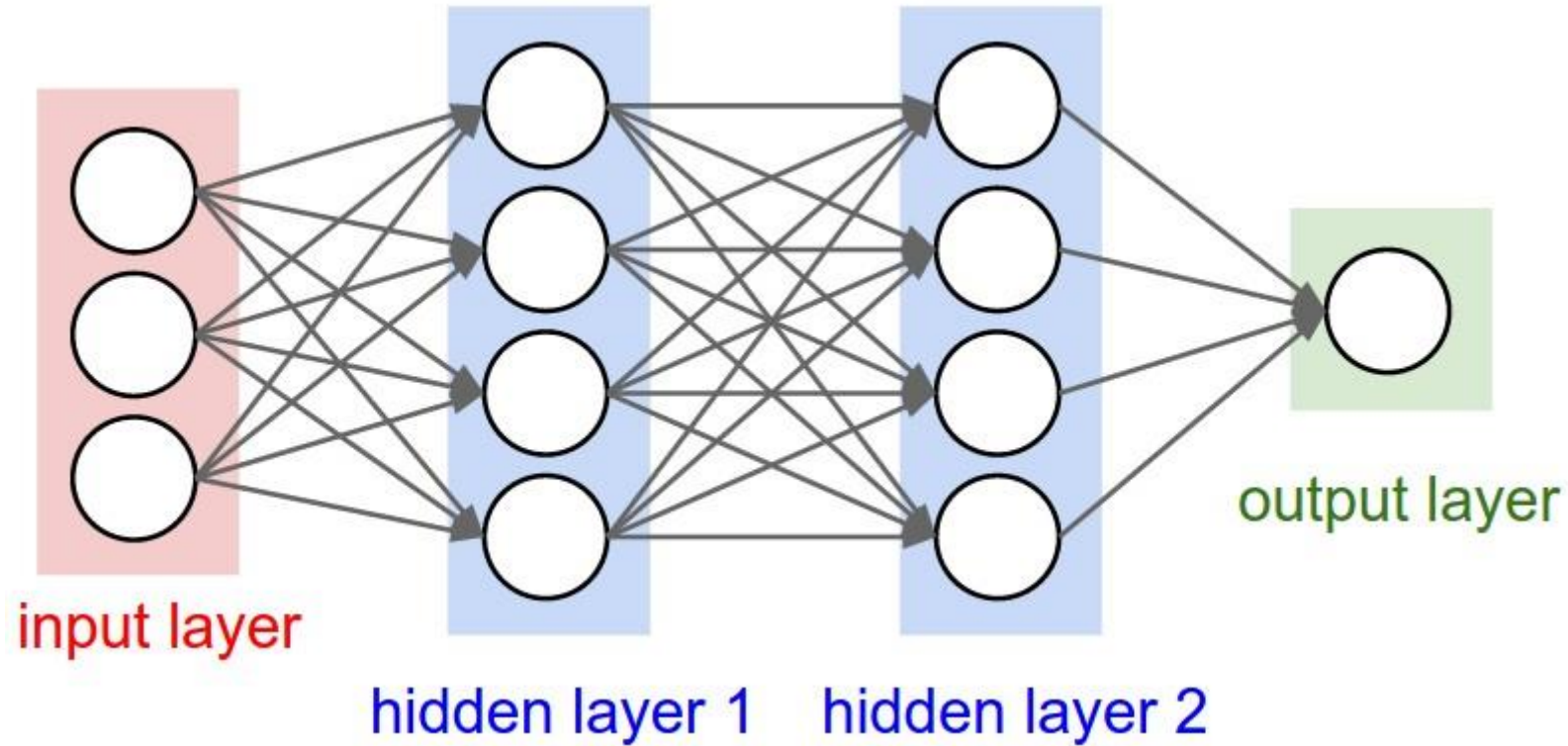
학습 - (2) Loss calculation



- 다중 class 분류의 경우

1.3 Training 4 steps in deep learning

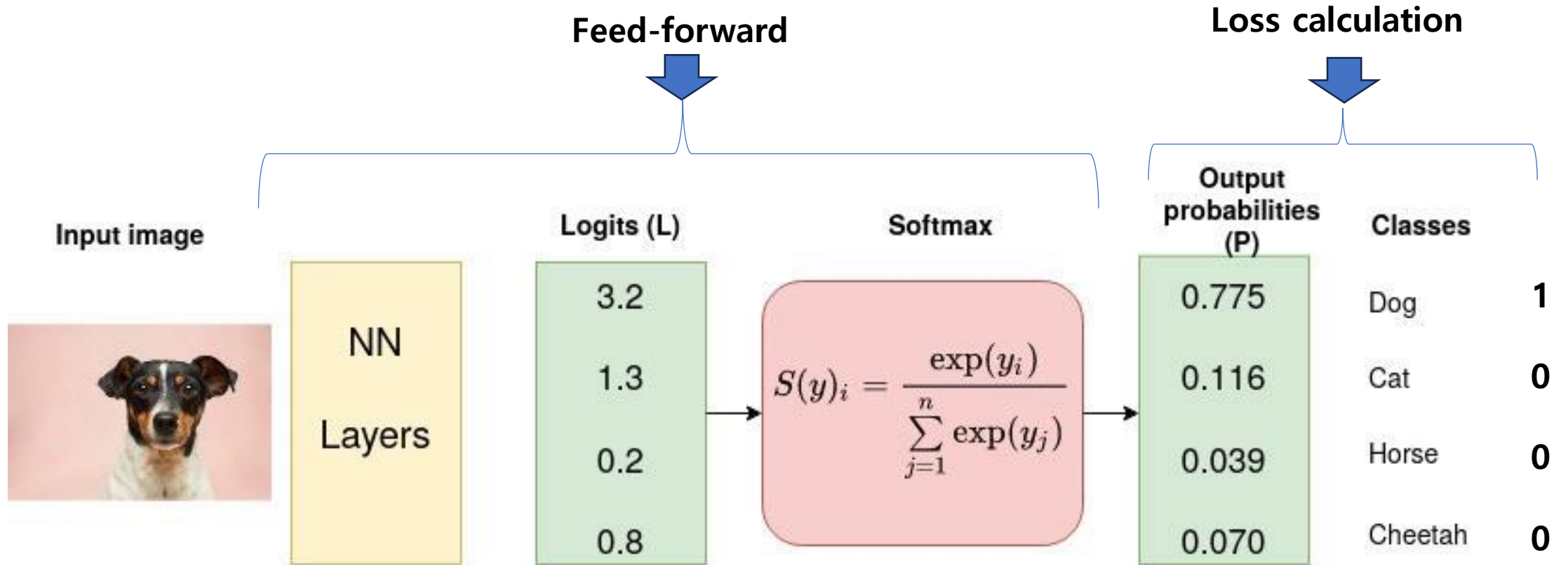
학습 - (2) Loss calculation



- 회귀 모델의 경우

1.3 Training 4 steps in deep learning

학습 – (2) Loss calculation



- 어떤 목적을 가지느냐에 따라 output node를 변경해주면 된다
- Cross-entropy loss, Hinge loss, L2 loss...

1.3 Training 4 steps in deep learning

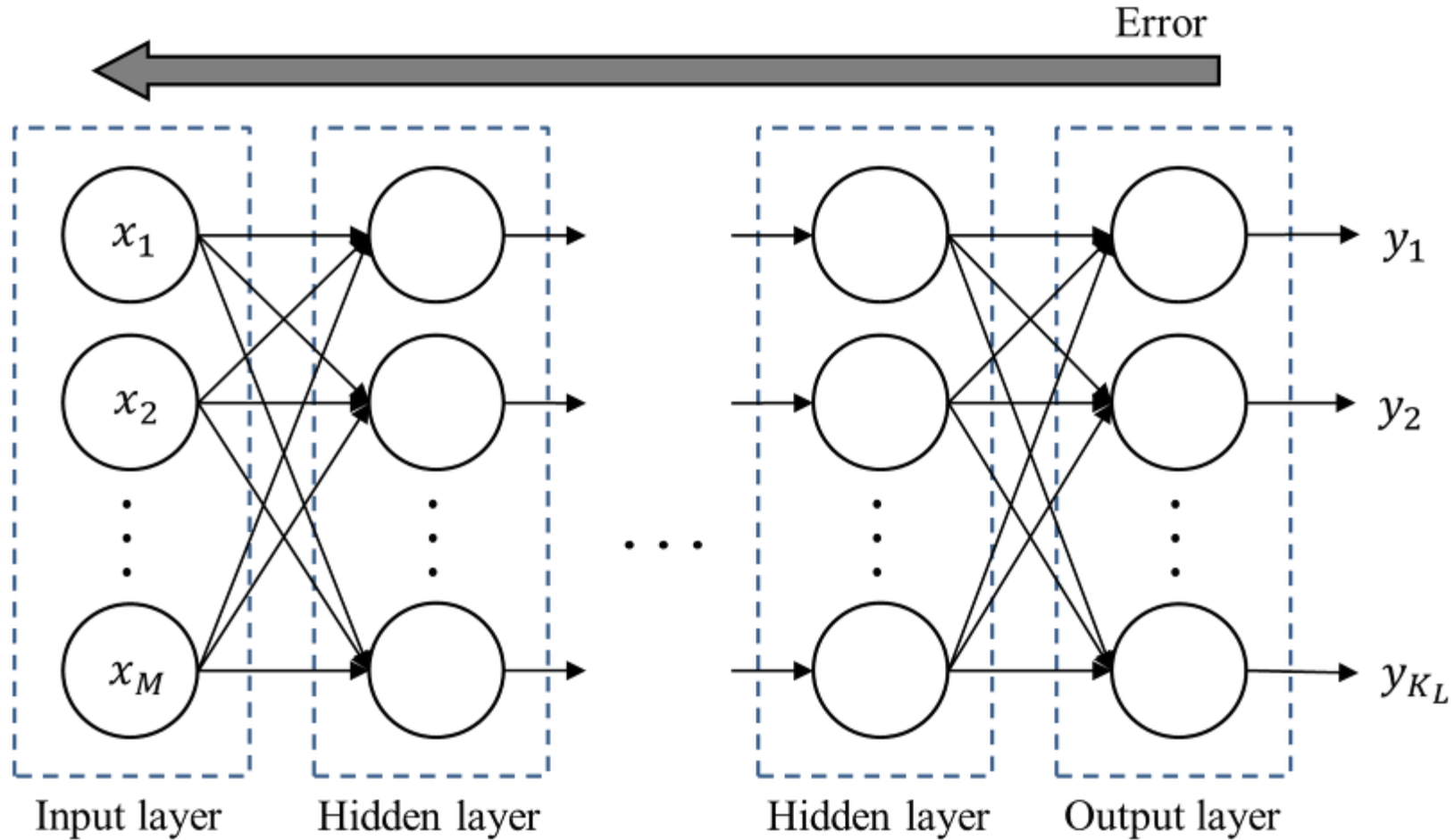
학습 – (3) Backpropagation

3. Backpropagation process:

- (Error) Backpropagation
- 오류 역전파라고 부르는 과정
- **신경망의 요소들이 얼마나 내가 원하는 정답 대비 에러를 발생시켰는지 계산**

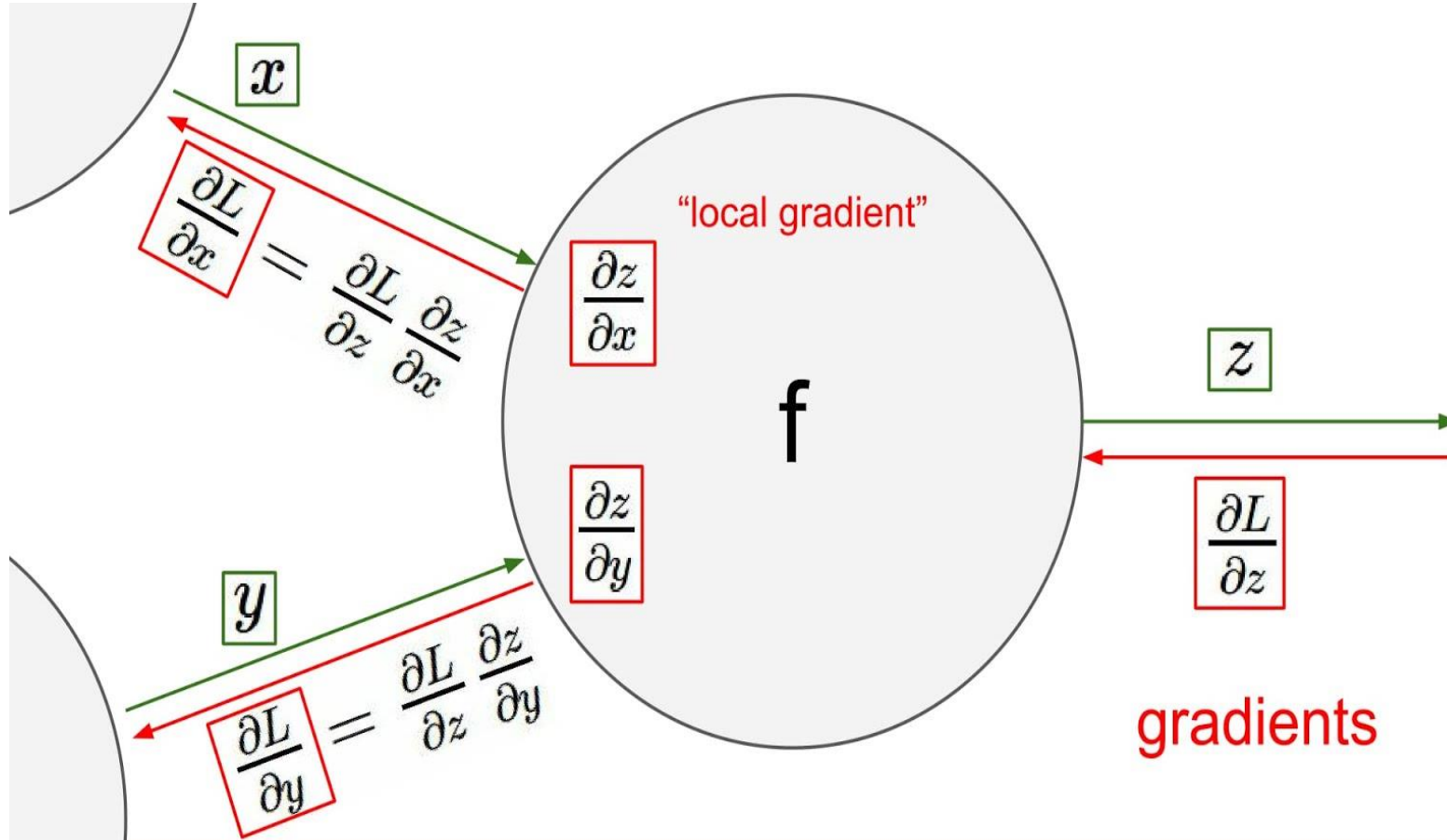
1.3 Training 4 steps in deep learning

학습 - (3) Backpropagation



1.3 Training 4 steps in deep learning

학습 - (3) Backpropagation



- 각 weight가 loss에 미치는 영향을 backpropagation step시 계산

1.3 Training 4 steps in deep learning

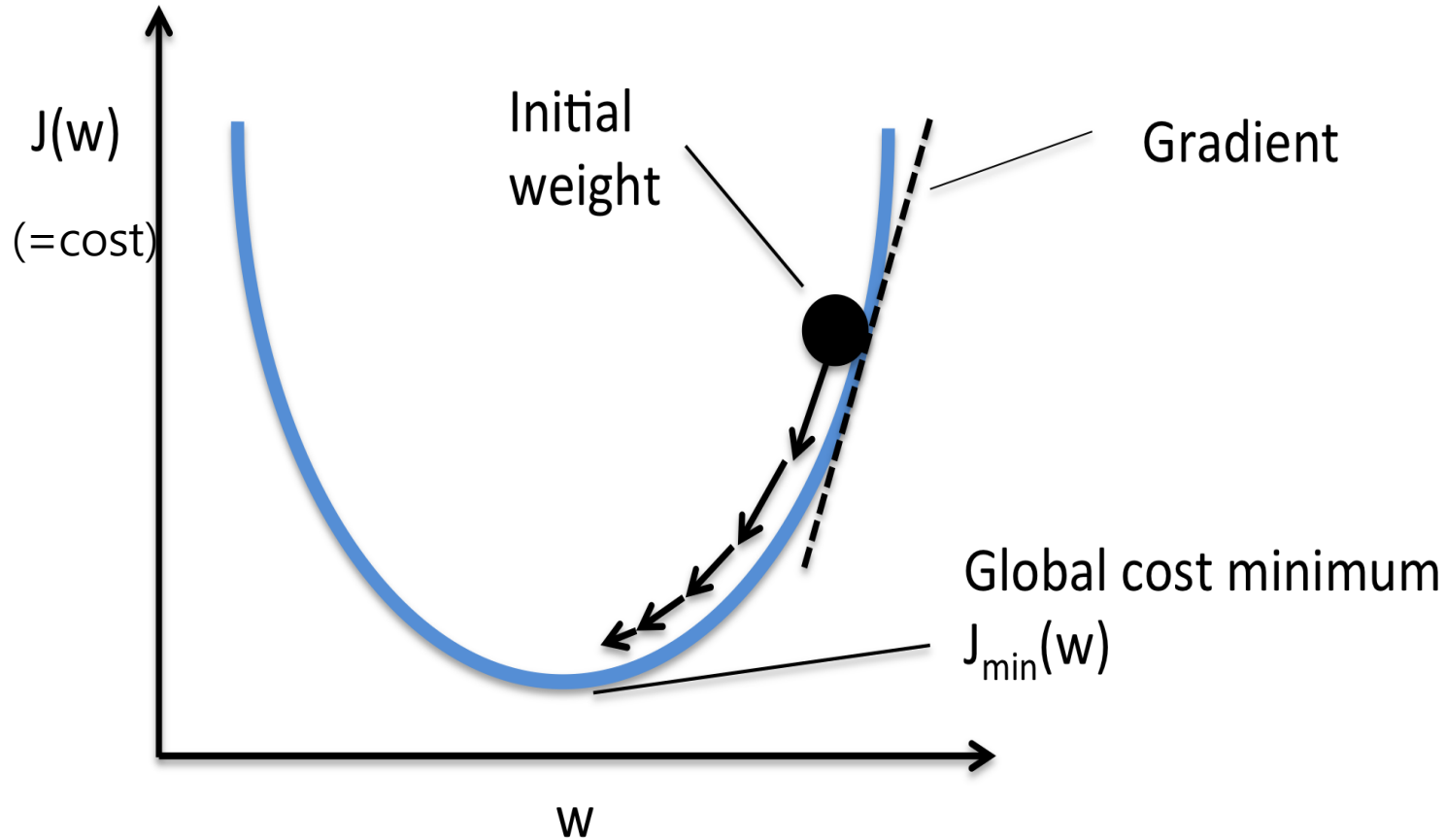
학습 - (4) Gradient update

4. Gradient update:

- 3단계에서 계산된 오류를 바탕으로 모델을 개선하는 과정.
- 모델의 오류 줄이는 방향으로 모델을 업데이트(갱신)
- 그렇다면 어떻게 모델을 업데이트 할것이야???

1.3 Training 4 steps in deep learning

학습 - (4) Gradient update



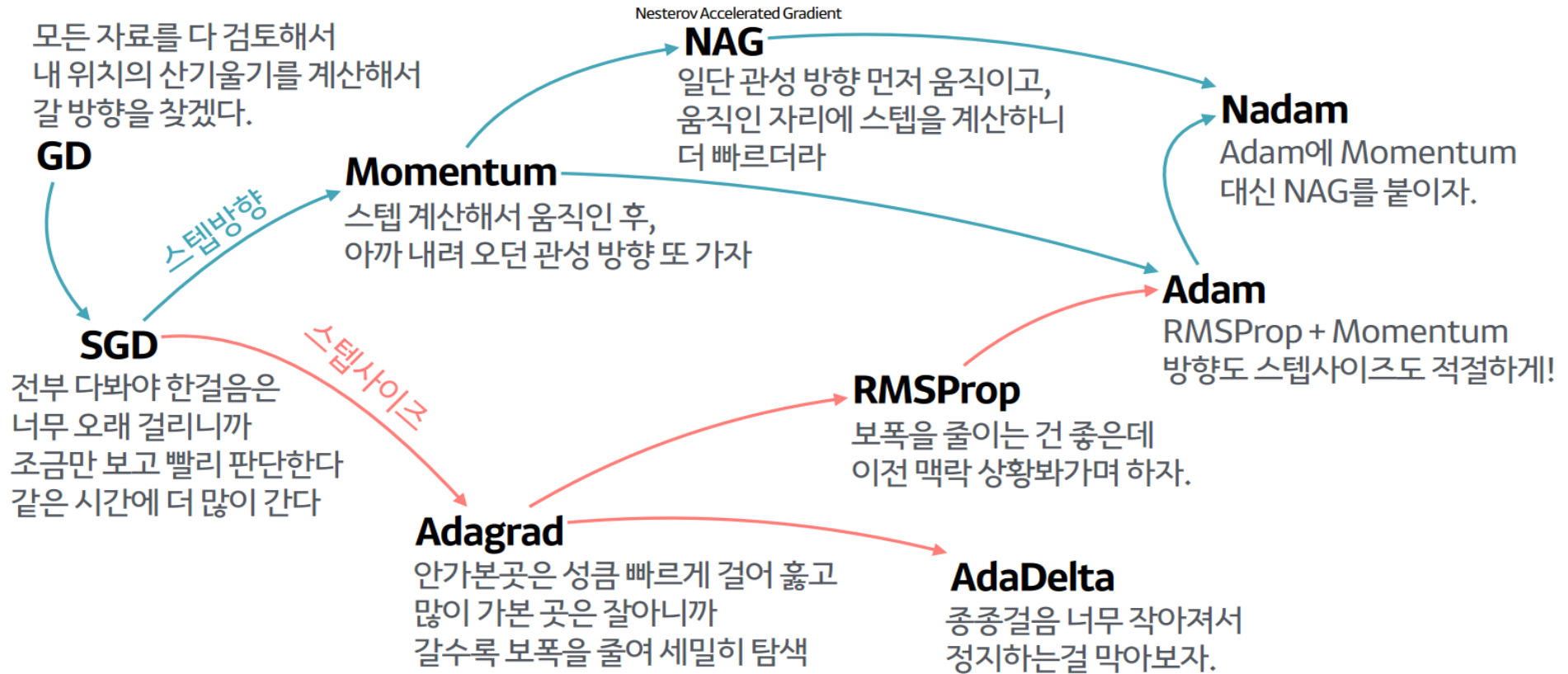
- 최적해를 찾기 위해서 반복해서 weight update
- 실질적으로는 NxM matrix의 요소들이 조금씩 변경되는 과정

1.3 Training 4 steps in deep learning

Optimizing algorithms

<https://www.slideshare.net/yongho/ss-79607172>

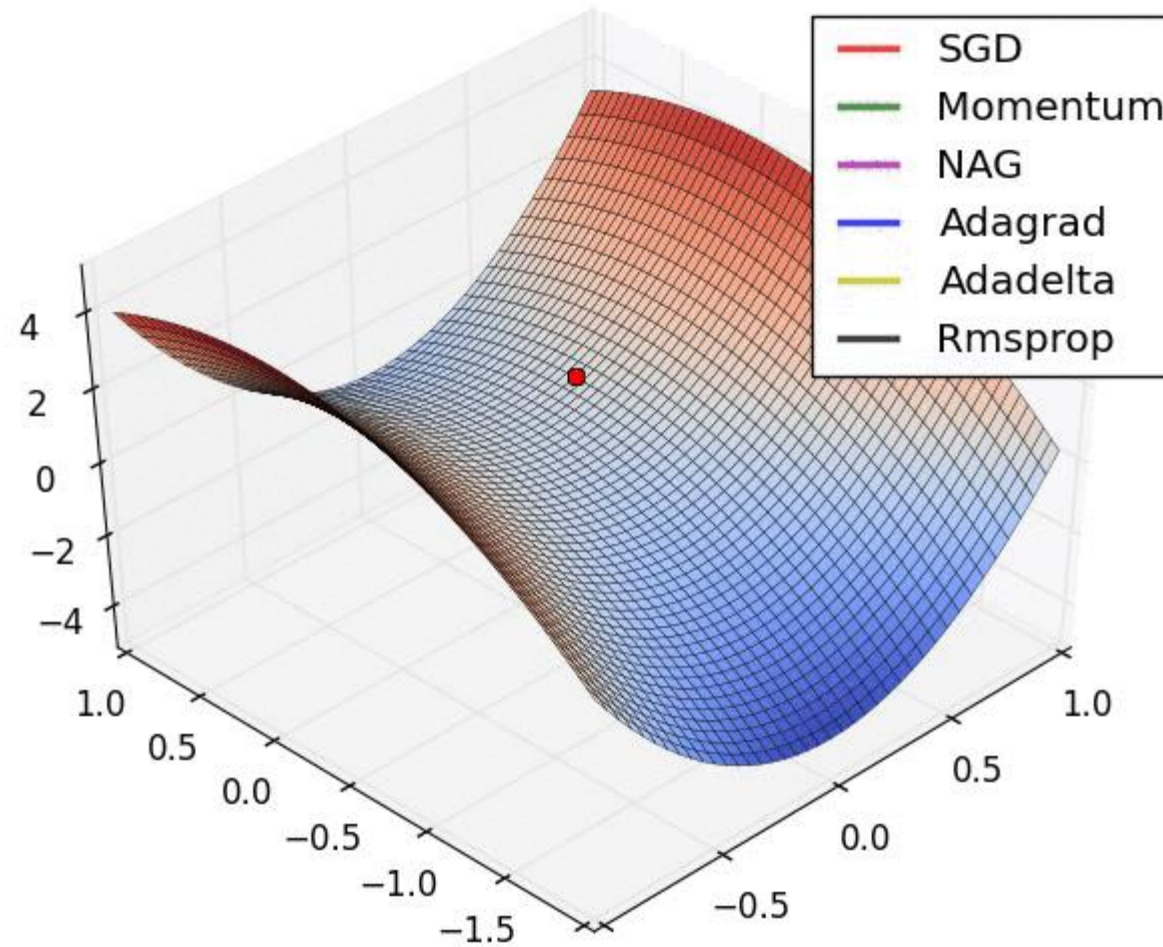
산 내려오는 작은 오솔길 찾기(Optimizer)의 발달 계보



<https://velog.io/@yookyungkho/%EB%94%A5%EB%9F%AC%EB%8B%9D%EC%98%B5%ED%8B%B0%EB%A7%88%EC%9D%B4%EC%A0%80-%EC%A0%95%EB%B3%B5%EA%B8%B0%EB%B6%80%EC%A0%9C-CS231n-Lecture7-Review>

1.3 Training 4 steps in deep learning

Optimizing algorithms



1.3 딥러닝 모델 학습의 4step

기본 신경망의 구성요소

1. 입력 레이어 (입력값)
2. 은닉 레이어
3. 출력 레이어 (출력값)
4. 가중치 (W)
5. 활성화 함수

1.3 파이썬을 이용한 신경망 구성 실습

문제 정의:

파이썬을 이용하여 3차원 데이터를 통한 이진 분류기 설계

데이터 및 모델 정의:

- Input data : 3차원
- 모델 구조: Input>layer1>layer2>output
- Output data : 0/1 예측 (이진분류)

```
import numpy as np
```

```
def sigmoid(x):
    return 1.0 / (1 + np.exp(-x))
```

```
def sigmoid_derivative(x):
    return x * (1.0 - x)
```

<보조 모듈>

시그모이드 함수와 그 도함수 정의:

- sigmoid: 입력값에 대한 시그모이드 함수를 정의합니다.
- sigmoid_derivative: 시그모이드 함수의 도함수를 정의합니다.

1.3 파이썬을 이용한 신경망 구성 실습

```
class NeuralNetwork:
    def __init__(self, x, y):
        self.input = x
        self.weights1 = np.random.rand(self.input.shape[1], 4)
        self.weights2 = np.random.rand(4, 1)
        self.y = y
        self.output = np.zeros(self.y.shape)

    def feedforward(self):
        self.layer1 = sigmoid(np.dot(self.input, self.weights1))
        self.output = sigmoid(np.dot(self.layer1, self.weights2))

    def backprop(self):
        loss = np.sum((self.y - self.output) ** 2)
        print(loss, 'loss 갱신')

        d_weights2 = np.dot(self.layer1.T, (2 * (self.y - self.output) *
            sigmoid_derivative(self.output)))
        d_weights1 = np.dot(self.input.T, (np.dot(2 * (self.y - self.output) *
            sigmoid_derivative(self.output), self.weights2.T) * sigmoid_derivative(self.layer1)))

        self.weights1 += d_weights1
        self.weights2 += d_weights2
        print(self.weights1, 'weight1 갱신')
        print(self.weights2, 'weight2 갱신')
```

Training 4-Step

1. Feed-forward
2. Loss calculation
3. Backpropagation
4. Gradient update

1.3 파이썬을 이용한 신경망 구성 실습

```
class NeuralNetwork:
    def __init__(self, x, y):
        self.input = x
        self.weights1 = np.random.rand(self.input.shape[1], 4)
        self.weights2 = np.random.rand(4, 1)
        self.y = y
        self.output = np.zeros(self.y.shape)

    def feedforward(self):
        self.layer1 = sigmoid(np.dot(self.input, self.weights1))
        self.output = sigmoid(np.dot(self.layer1, self.weights2))

    def backprop(self):
        loss = np.sum((self.y - self.output) ** 2)
        print(loss, 'loss 갱신')

        d_weights2 = np.dot(self.layer1.T, (2 * (self.y - self.output) *
            sigmoid_derivative(self.output)))
        d_weights1 = np.dot(self.input.T, (np.dot(2 * (self.y - self.output) *
            sigmoid_derivative(self.output), self.weights2.T) * sigmoid_derivative(self.layer1)))

        self.weights1 += d_weights1
        self.weights2 += d_weights2
        print(self.weights1, 'weight1 갱신')
        print(self.weights2, 'weight2 갱신')
```

Training 4-Step

1. Feed-forward
2. Loss calculation
3. Backpropagation
4. Gradient update

1.3 파이썬을 이용한 신경망 구성 실습

```
class NeuralNetwork:
    def __init__(self, x, y):
        self.input = x
        self.weights1 = np.random.rand(self.input.shape[1], 4)
        self.weights2 = np.random.rand(4, 1)
        self.y = y
        self.output = np.zeros(self.y.shape)
```

```
def feedforward(self):
    self.layer1 = sigmoid(np.dot(self.input, self.weights1))
    self.output = sigmoid(np.dot(self.layer1, self.weights2))
```

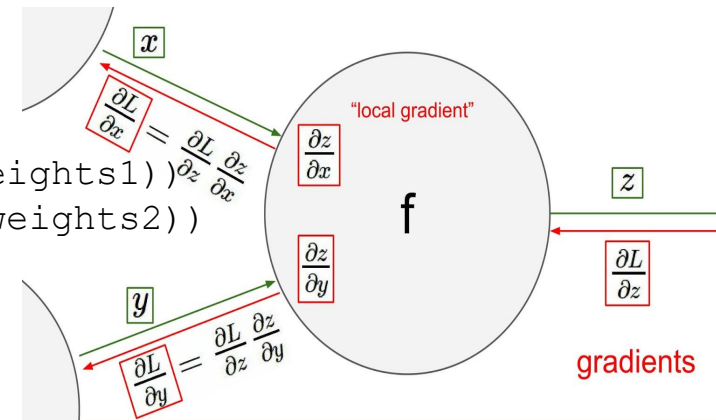
```
def backprop(self):
    loss = np.sum((self.y - self.output) ** 2)
    print(loss, 'loss 갱신')
```

```
d_weights2 = np.dot(self.layer1.T, (2 * (self.y - self.output) *
    sigmoid_derivative(self.output)))
d_weights1 = np.dot(self.input.T, (np.dot(2 * (self.y - self.output) *
    sigmoid_derivative(self.output), self.weights2.T) * sigmoid_derivative(self.layer1)))
```

```
self.weights1 += d_weights1
self.weights2 += d_weights2
print(self.weights1, 'weight1 갱신')
print(self.weights2, 'weight2 갱신')
```

Training 4-Step

1. Feed-forward
2. Loss calculation
3. Backpropagation
4. Gradient update



1.3 파이썬을 이용한 신경망 구성 실습

```
class NeuralNetwork:
    def __init__(self, x, y):
        self.input = x
        self.weights1 = np.random.rand(self.input.shape[1], 4)
        self.weights2 = np.random.rand(4, 1)
        self.y = y
        self.output = np.zeros(self.y.shape)

    def feedforward(self):
        self.layer1 = sigmoid(np.dot(self.input, self.weights1))
        self.output = sigmoid(np.dot(self.layer1, self.weights2))

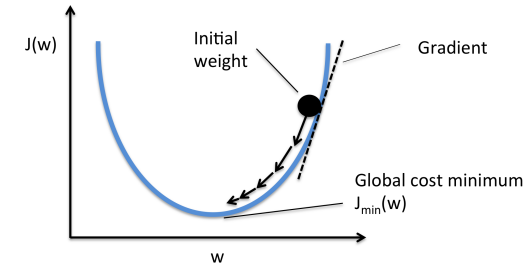
    def backprop(self):
        loss = np.sum((self.y - self.output) ** 2)
        print(loss, 'loss 갱신')

        d_weights2 = np.dot(self.layer1.T, (2 * (self.y - self.output) *
            sigmoid_derivative(self.output)))
        d_weights1 = np.dot(self.input.T, (np.dot(2 * (self.y - self.output) *
            sigmoid_derivative(self.output), self.weights2.T) * sigmoid_derivative(self.layer1)))

        self.weights1 += d_weights1
        self.weights2 += d_weights2
        print(self.weights1, 'weight1 갱신')
        print(self.weights2, 'weight2 갱신')
```

Training 4-Step

1. Feed-forward
2. Loss calculation
3. Backpropagation
4. Gradient update



1.3 파이썬을 이용한 신경망 구성 실습

```
if __name__ == "__main__":  
    X = np.array([[0, 0, 1],  
                  [0, 1, 1],  
                  [1, 0, 1],  
                  [1, 1, 1]])  
    y = np.array([[0], [1], [1], [0]])  
  
    nn = NeuralNetwork(X, y)  
    for i in range(10000):  
        nn.feedforward()  
        nn.backprop()
```

- 실제 실행 코드

1.3 파이썬을 이용한 신경망 구성 실습

```
0.00827865525880996 loss 갱신
[[-2.28801603  5.39093776  6.56727789  0.41866086]
 [ 6.2442089   5.17124794 -3.95818602  1.20904688]
 [-0.44842993 -0.71366403  1.09951768 -0.70534047]] weight1 갱신
[[-6.50481787]
 [10.12071937]
 [-6.55124428]
 [-0.28015389]] weight2 갱신
```

...

```
0.004554957569288448 loss 갱신
[[-2.46892479  5.52997304  6.88164238  0.43504603]
 [ 6.59626589  5.33445894 -4.12369794  1.23312642]
 [-0.44274222 -0.71574666  1.09804955 -0.77787178]] weight1 갱신
[[-7.06906951]
 [10.95224396]
 [-7.11443184]
 [-0.27201803]] weight2 갱신
```

⋮

최종 예측값

```
0.003919446030090292 loss 갱신
[[-2.51244309  5.56341088  6.95729744  0.43951321]
 [ 6.68021863  5.37337985 -4.1632904   1.23903766]
 [-0.44106459 -0.71572005  1.09764057 -0.7957777 ]] weight1 갱신
[[-7.2104569 ]
 [11.16208644]
 [-7.25582749]
 [-0.27179014]] weight2 갱신
[[0.0091718 ]
 [0.966804  ]
 [0.96616527]
 [0.03985675]] nn.output
```

1.3 케라스로 신경망 만들기



1.3 케라스로 신경망 만들기

Overview

- 문제 정의: 포유류/비포유류를 분류하는 분류기 설계
- 데이터 정의: X =동물의 특징 3가지 Y =포유류(0)/비포유류(1)
- 데이터 개수: 학습데이터 4개 / 테스트 데이터 2개
- 학습 신경망 정의: 2개 층으로 이루어진 multi-layer

1.3 케라스로 신경망 만들기

1. Module import 및 data 정의

```
# 필요한 라이브러리 импорт
import numpy as np
from keras.models import Sequential
from keras.layers import Dense

# 샘플 데이터 생성
# 특징 변수: 다리의 개수, 눈의 개수, 꼬리의 길이
# 이진 클래스: 포유류인지 아닌지 (1: 포유류, 0: 비포유류)
X = np.array([[4, 2, 1.5],
               [2, 2, 0.5],
               [4, 2, 2.0],
               [0, 8, 0.1]])

y = np.array([1, 0, 1, 0])

Test_X = np.array([[2, 4, 1.0],
                    [4, 6, 2.5]])
Test_y = np.array([0, 1])
```

1.3 케라스로 신경망 만들기

2. 모델 정의 및 학습

모델 정의

```
model = Sequential()
```

```
model.add(Dense(10, input_dim=3, activation='relu')) # 입력 레이어, 특징 변수의 개수  
#에 맞게 설정
```

```
model.add(Dense(1, activation='sigmoid')) # 출력 레이어, 이진 분류이므로 시그모이드  
#활성화 함수 사용
```

모델 컴파일

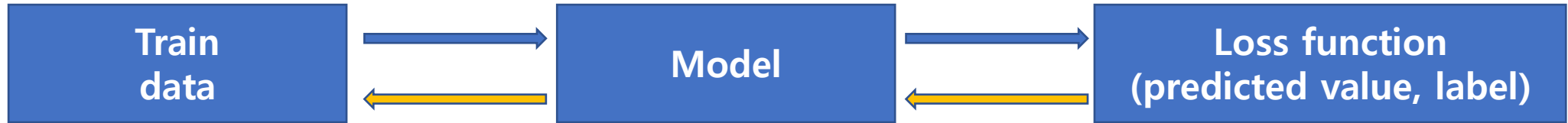
```
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

모델 학습

```
model.fit(X, y, epochs=100, batch_size=1, verbose=1)
```

1.3 케라스로 신경망 만들기

<학습과정>



<추론과정>

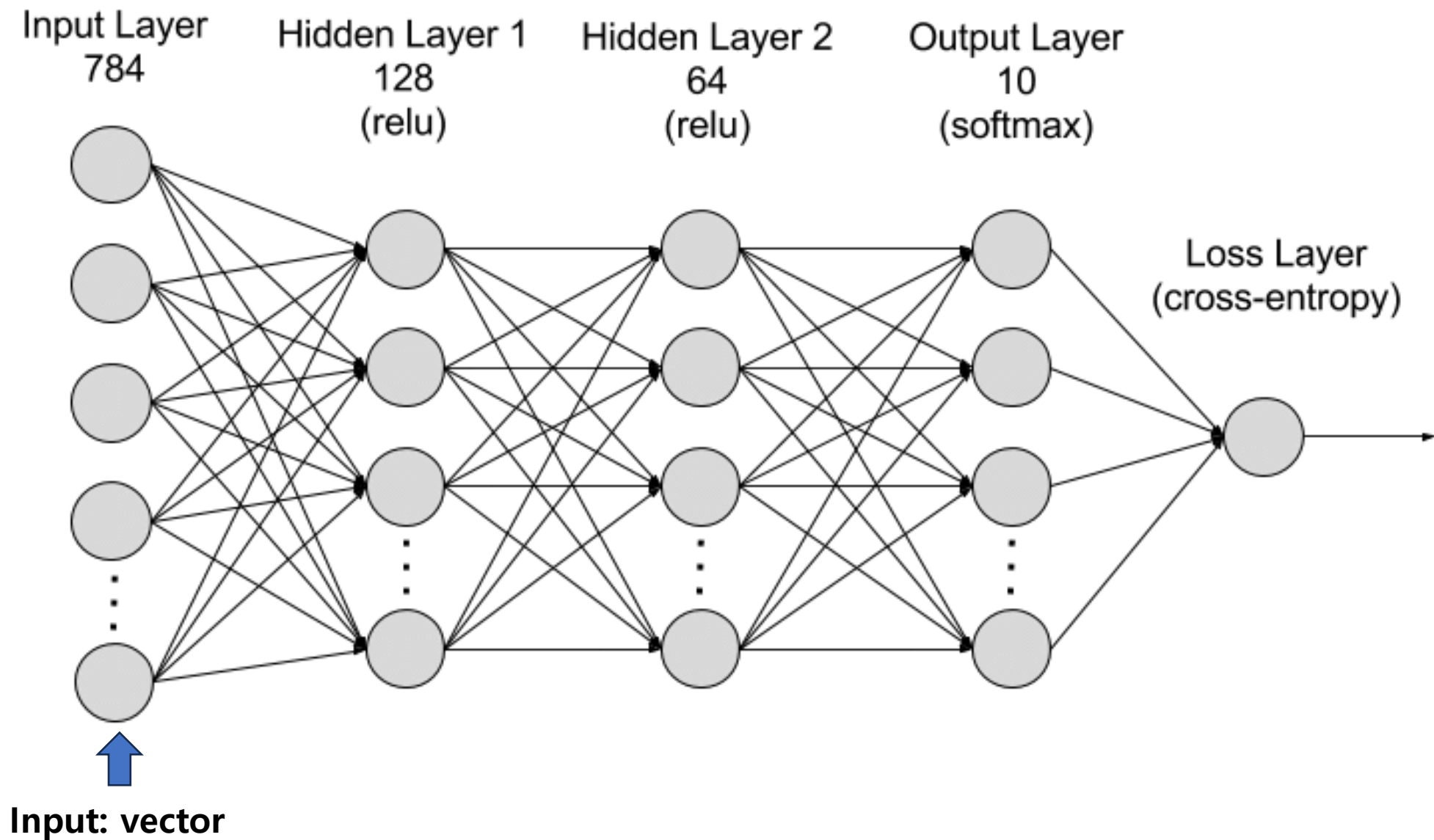


3. 추론 과정

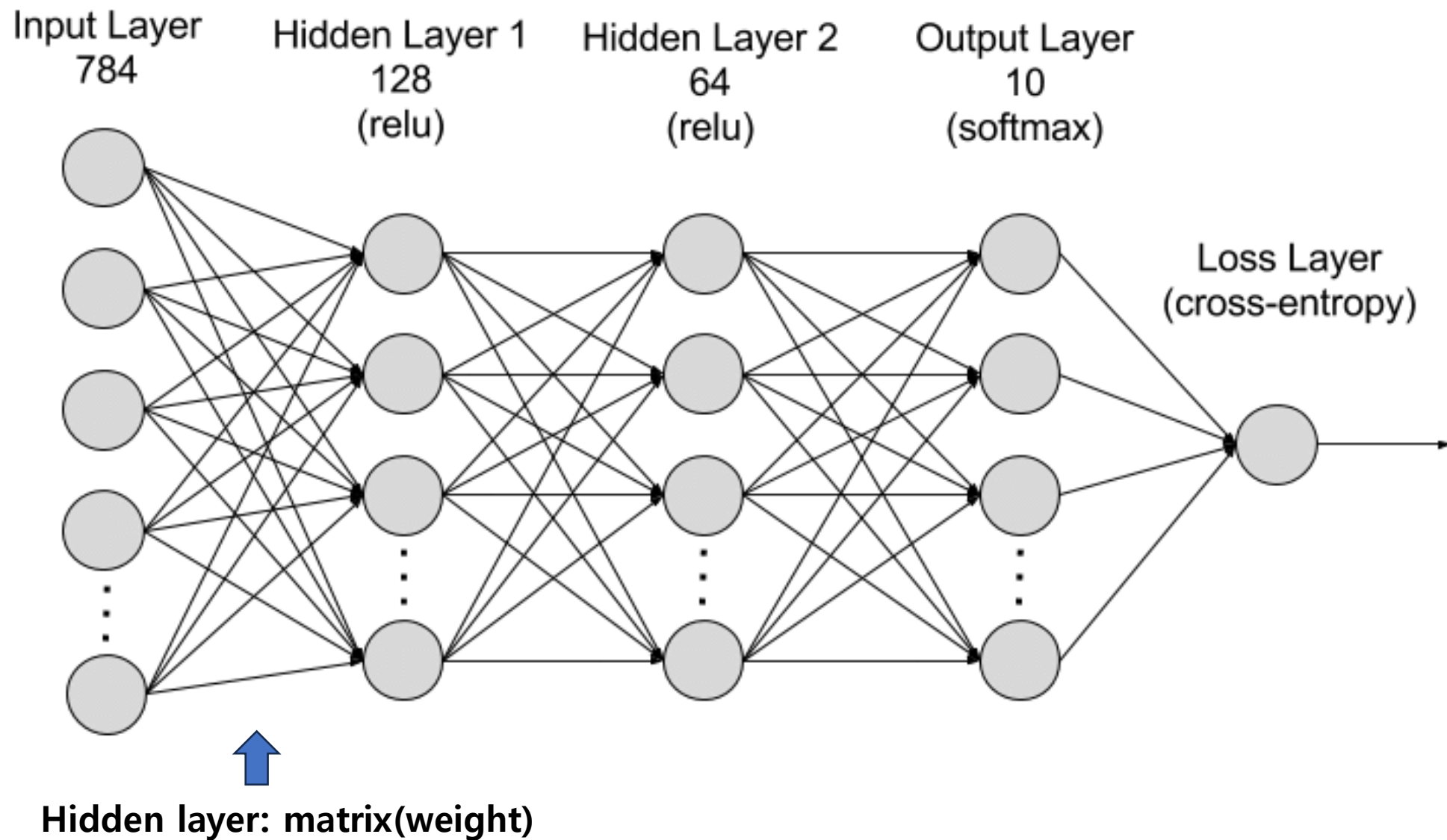
```
# 학습된 모델을 사용하여 예측
predictions = model.predict(Test_X)
predictions = np.round(predictions) # 확률값을 이진으로 변환

# 예측 결과 출력
print("Predictions:")
print(predictions)
```

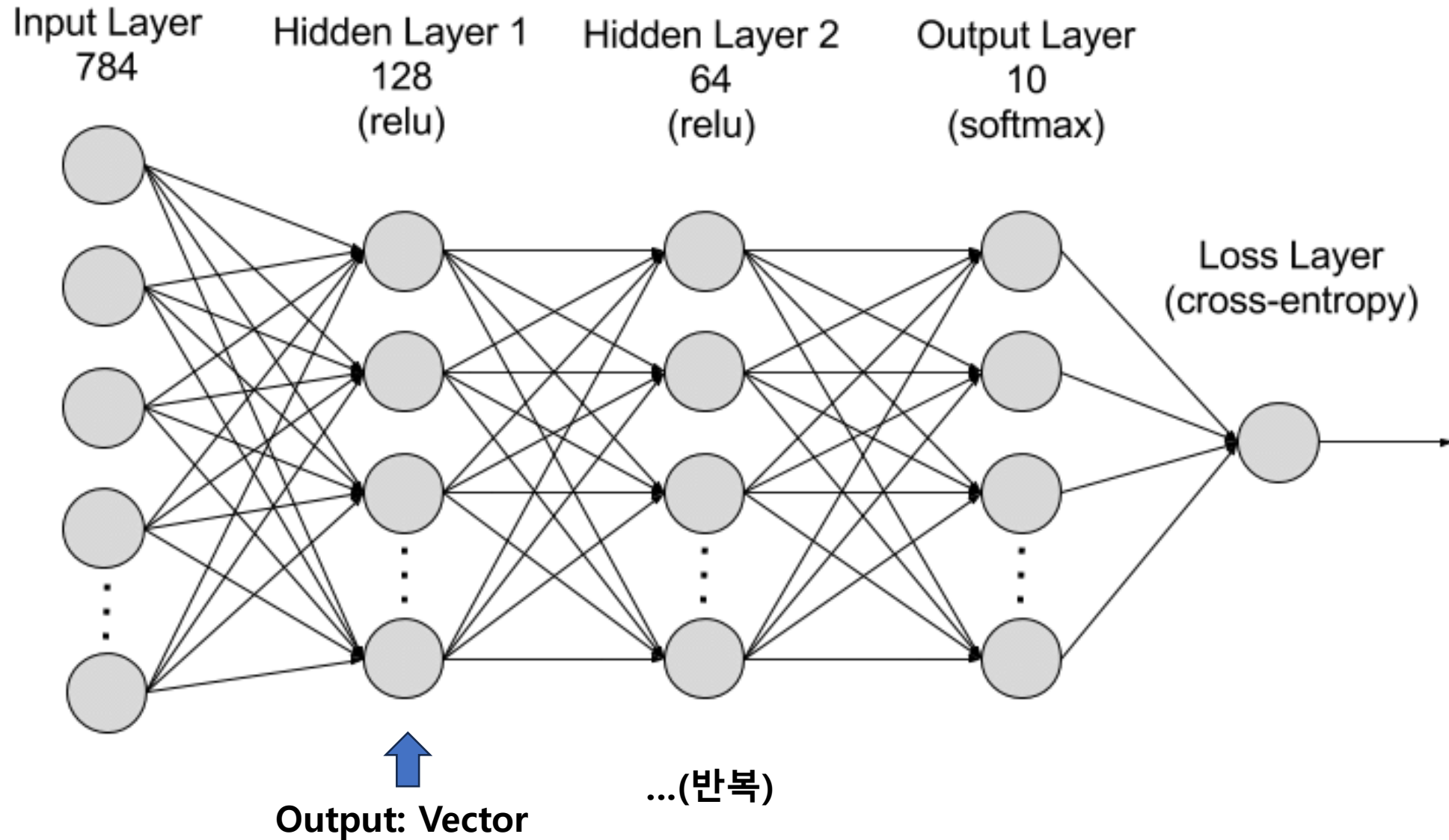
딥러닝을 위한 기초 수학



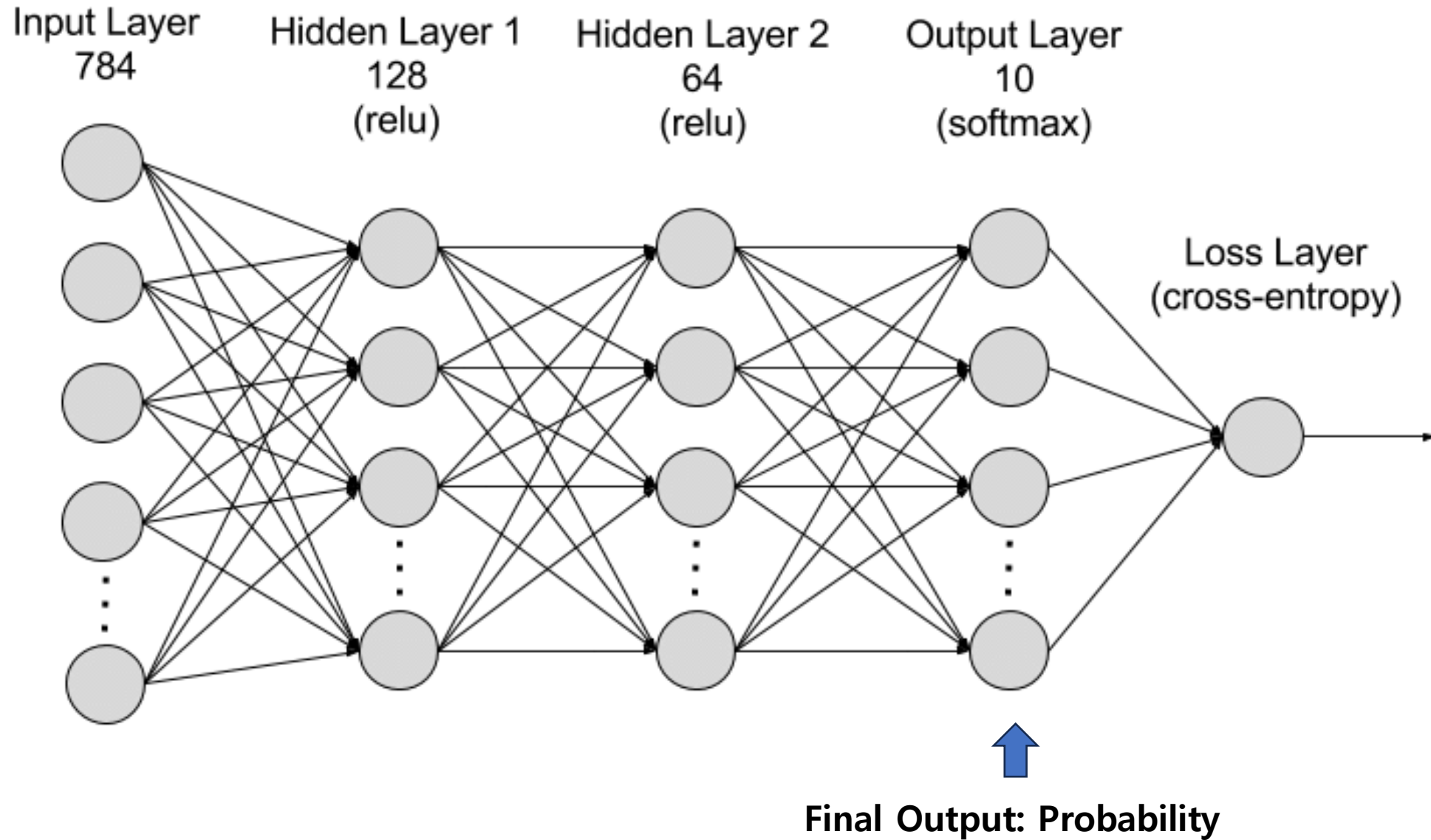
딥러닝을 위한 기초 수학



딥러닝을 위한 기초 수학



딥러닝을 위한 기초 수학



감사합니다